Session 18 Homework — A New Index, an Atomic Transfer, and a Look Ahead

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

By the end of class, a real index exists (`idx_issues_status`), a real atomic transaction pattern is proven for generating issue keys, and your team built a new real table (`activity_logs`) with its own compound indexes and a real, proven `SELECT ... FOR UPDATE` lock. Nobody has asked you yet to choose and build your own index for a real question you identify yourself, or to combine a transaction with an existing constraint you already know, across two real tables, in one atomic unit.

Goal

An extended, saved `sql-practice/queries.sql` (the same file Session 17 started), containing: a new, independently-chosen compound index proven with a real bulk-seeded dataset and `EXPLAIN ANALYZE` (CORE); a real, saved, atomic transaction correctly transferring real workspace ownership across two tables (STRETCH); and a short, real, written reflection on what a schema migration tool would automate, placed above one final proof query (CHALLENGE).

What to build

You already know `CREATE INDEX`, `BEGIN` / `COMMIT` / `ROLLBACK`, and `UNIQUE` constraints. Research nothing new — apply what you already have:

- A real question: "every real comment written by a given real user, newest first." Bulk-seed enough real comments (skewed, so one real user has commented rarely) to make the question genuinely slow without an index, prove it with `EXPLAIN ANALYZE`, then build the index that fixes it, and prove that too. Save the final query.
- A real transaction transferring one real workspace's ownership from its current owner to another real, existing member: both the `owner_id` change on `workspaces` and that member's own role change (to `'owner'`) on `workspace_members` must happen together, inside one transaction, or not at all. Think carefully about whether the new owner already has a membership row — what happens if you try to insert a second one for them?
- A short, real, written answer (see CHALLENGE), placed directly above a final query, in `sql-practice/queries.sql` — the same convention Session 17's own homework already used.

CORE (required)

- Bulk-seed `comments` with a real, skewed distribution (most comments from a few frequent users, a handful from one rare user) and prove, with `EXPLAIN ANALYZE`, that the question "every comment by the rare user, newest first" currently forces a full scan.

- Build and prove a compound index that fixes it. Save the final query to `sql-practice/queries.sql`.

STRETCH (required)

A real, saved transaction in `sql-practice/queries.sql`: transfer a real workspace's ownership to another real, already-existing member, correctly and atomically (both tables' own changes happen together). Think about what goes wrong if you write this the naive way.

CHALLENGE (required)

Write a short, direct, written answer, placed directly above a final query, in `sql-practice/queries.sql`:

Across this session and the last, you've written `schema.sql` by hand, re-run it by hand to reset practice databases, and manually kept every `ON DELETE`/index/transaction decision consistent in your head across seven real tables. Name two concrete things a schema migration tool (an ORM) would automate that you just did by hand this block, and one concrete risk of relying on manual SQL file re-runs across a team of developers instead. Then write one final query: which five real `activity_logs` entries are most recent for a given real workspace?

Rules

Allowed: everything already shown live or in teamwork — `CREATE TABLE`, `CREATE INDEX`, `PRIMARY KEY`, `FOREIGN KEY`/`REFERENCES`, `UNIQUE`, `CHECK`, `INSERT`, `SELECT`, `WHERE`, `ORDER BY`, `JOIN`, `LEFT JOIN`, `GROUP BY`, aggregate functions, `BEGIN`/`COMMIT`/`ROLLBACK`, `EXPLAIN ANALYZE`, plain SQL comments (`--`).

Not yet: no `SELECT ... FOR UPDATE` is required for this homework (that stays teamwork's own material, already done); no naming a specific schema migration tool or its syntax in your written CHALLENGE answer — describe what a migration tool would do, never name one.

Test it

Re-seed a fresh database before testing — teamwork's own delete proofs deliberately removed real rows, so a database left over from class will not match the expected results below.

macOS terminal

BASH

```
createdb b4f_board_sql_practice_hw   # or drop/recreate your own practice database
psql -d b4f_board_sql_practice_hw
```

inside psql — b4f_board_sql_practice_hw=>

TEXT

```
\i sql-practice/schema.sql
```

SQL

```
-- re-run the seed INSERT statements from this session's own live-code and teamwork steps,
in order
```

TEXT

```
-- then run your own saved queries file end to end
\i sql-practice/queries.sql
```

Stay inside this one psql session for both files — no need to reconnect between them.

Done when

- `sql-practice/queries.sql` runs cleanly, end to end, against a freshly re-seeded database, with zero errors.
- The CORE index is proven, with real `EXPLAIN ANALYZE` output, to actually be used, and the saved query returns the correct real rows.
- The STRETCH transaction correctly updates both tables together, atomically.
- Your CHALLENGE written answer gives two separate, concrete things done by hand this block (not a generic explanation of why ORMs are good) and names no specific tool, and the final query correctly returns five real, ordered `activity_logs` rows.

Working time

About 60–80 minutes — the index work reuses concepts already shown live; the transaction integration and the written reflection both deserve real thinking time.